

Лабораторная работа по теме №2 "Захват звука.

Библиотека miniaudio"

Цель работы: В лабораторной работе предлагается изучить возможности захвата звука. В качестве инструмента при выполнении лабораторной работы можно использовать любую систему программирования, позволяющую считывать и выводить файлы. Дается шаблон программы на языке C++.

Теоретическое обоснование

Библиотека **miniaudio** включает как низкоуровневые, так и высокоуровневые API. Низкоуровневый API хорош для тех, кто хочет выполнять все микширование самостоятельно и требует только легковесный интерфейс к базовому аудиоустройству. Высокоуровневый API подходит для тех, у кого есть сложные требования к микшированию и эффектам.

В miniaudio объекты являются прозрачными структурами. В отличие от многих других библиотек, нет дескрипторов к непрозрачным объектам, что означает, что вам нужно самостоятельно выделять память для объектов. В примерах, представленных в этих методических указаниях, вы часто увидите объекты, объявленные в стеке. Вам нужно быть осторожными при переносе этих примеров в ваш собственный код, чтобы не объявить случайно ваши объекты в стеке и затем не сделать их недействительными после возврата функции. Кроме того, вы должны гарантировать, что адрес памяти ваших объектов остается одним и тем же на протяжении их жизни. Поэтому вы не можете делать копии ваших объектов.

Для использования miniaudio просто включите "miniaudio.h" как любой другой заголовочный файл и добавьте "miniaudio.c" в ваше дерево исходных файлов. Если вы не хотите добавлять его в дерево исходных файлов, вы можете скомпилировать и связать его как любую другую библиотеку. Обратите внимание, что совместимость ABI не гарантируется между версиями, даже с исправлениями ошибок, поэтому будьте осторожны при компиляции как общий объект.

Шаблон конфигурация/инициализация используется во всей библиотеке. Идея заключается в том, что вы настраиваете объект конфигурации и передаете его в процедуру инициализации. Преимущество этой системы в том, что объект конфигурации может быть инициализирован логическими значениями по умолчанию, и к нему могут быть добавлены

новые свойства без нарушения API. Объект конфигурации может быть размещен в стеке и не нуждается в поддержке после инициализации соответствующего объекта.

Низкоуровневый API

Низкоуровневый API дает вам доступ к необработанным аудиоданным аудиоустройства. Он поддерживает воспроизведение, захват, полнодуплексную связь и loopback (только WASAPI). Вы можете перечислить устройства, чтобы определить, к каким физическим устройствам вы хотите подключиться.

Низкоуровневый API использует концепцию "устройства" как абстракцию для физических устройств. Идея состоит в том, что вы выбираете физическое устройство для вывода или захвата аудио, а затем перемещаете данные к/от устройства, когда miniaudio говорит вам об этом. Данные доставляются к устройствам и от них асинхронно через callback, который вы указываете при инициализации устройства.

При инициализации устройства сначала нужно его настроить. Конфигурация устройства позволяет указать такие вещи, как формат данных, доставляемых через callback, размер внутреннего буфера и ID устройства, от которого вы хотите выводить или захватывать аудио.

После настройки конфигурации устройства вы можете инициализировать устройство. При инициализации устройства нужно заранее выделить память для объекта устройства. Это дает приложению полный контроль над тем, как выделяется память. В приведенном ниже примере мы инициализируем устройство воспроизведения в стеке, но вы можете выделить его в куче, если это лучше подходит для вашей ситуации.

```
void data_callback(ma_device* pDevice, void* pOutput, const void* pInput,
ma_uint32 frameCount)
{
    // В режиме воспроизведения данные копируются в pOutput. В режиме захвата
    данные считываются из pInput. В смешанном режиме – и то и другое.
    // pOutput и pInput будут доступны и вы сможете передавать данные напрямую
    из pInput в pOutput, но не более чем frameCount кадров.
}

int main()
{
    ma_device_config config = ma_device_config_init(ma_device_type_playback);
    config.playback.format = ma_format_f32; // Set to ma_format_unknown to
use the device's native format.
    config.playback.channels = 2; // Set to 0 to use the device's
native channel count.
    config.sampleRate = 48000; // Set to 0 to use the device's
native sample rate.
    config.dataCallback = data_callback; // This function will be called
when miniaudio needs more data.
    config.pUserData = pMyCustomData; // Can be accessed from the
device object (device.pUserData).
    ma_device device;
    if (ma_device_init(NULL, &config, &device) != MA_SUCCESS) {
        return -1; // Ошибка инициализации устройства.
    }
}
```

```

    ma_device_start(&device);    // По-умолчанию устройство находится в режиме
    ожидания, поэтому необходимо пробуждать его самостоятельно.
    // Основной код вашей программы.
    ma_device_uninit(&device);
    return 0;
}

```

В примере выше `data_callback()` - это место, где аудиоданные записываются и считываются с устройства. Идея в режиме воспроизведения заключается в том, что вы заставляете звук издаваться из динамиков, записывая аудиоданные в выходной буфер (`pOutput` в примере). В режиме захвата вы считываете данные из входного буфера (`pInput`) для извлечения звука, захваченного микрофоном. Параметр `frameCount` сообщает вам, сколько кадров можно записать в выходной буфер и прочитав из входного буфера. "Кадр" — это один образец для каждого канала. Например, в стереопотоке (2 канала) один кадр состоит из 2 образцов: один для левого, один для правого. Количество каналов определяется конфигурацией устройства. Размер в байтах отдельного образца определяется форматом образца, который также указывается в конфигурации устройства. Многоканальные аудиоданные всегда чередуются, что означает, что образцы для каждого кадра хранятся рядом друг с другом в памяти.

Конфигурация устройства определяется структурой `ma_device_config`. Объект конфигурации всегда инициализируется с помощью `ma_device_config_init()`. Важно всегда инициализировать конфигурацию этой функцией, поскольку она инициализирует её логическими значениями по умолчанию и гарантирует, что ваша программа не сломается при добавлении новых членов в структуру `ma_device_config`.

Пример использует довольно простую и стандартную конфигурацию устройства. Вызов `ma_device_config_init()` принимает один параметр, который указывает, является ли устройство устройством воспроизведения, захвата, дуплексным или `loopback` (`loopback` устройства поддерживаются не всеми бэкендами).

Член `config.playback.format` устанавливает формат образца, который может быть одним из следующих (все форматы в нативном порядке байтов):

Символ	Описание	Диапазон
<code>ma_format_f32</code>	32-битное число с плавающей точкой	<code>[-1, 1]</code>
<code>ma_format_s16</code>	16-битное знаковое целое	<code>[-32768, 32767]</code>
<code>ma_format_s24</code>	24-битное знаковое целое (упакованное)	<code>[-8388608, 8388607]</code>
<code>ma_format_s32</code>	32-битное знаковое целое	<code>[-2147483648, 2147483647]</code>
<code>ma_format_u8</code>	8-битное беззнаковое целое	<code>[0, 255]</code>

Член `config.playback.channels` устанавливает количество каналов для использования с устройством. Количество каналов не может превышать `MA_MAX_CHANNELS`. Член `config.sampleRate` устанавливает частоту дискретизации (которая должна быть одинаковой

для воспроизведения и захвата в полнодуплексных конфигурациях). Обычно это устанавливается в 44100 или 48000, но может быть установлено в любое значение. Рекомендуется держать это между 8000 и 384000.

Обратите внимание, что оставление формата, количества каналов и/или частоты дискретизации в их значениях по умолчанию приведет к использованию нативной конфигурации внутреннего устройства, что полезно, если вы хотите избежать накладных расходов на автоматическое преобразование данных `miniaudio`.

В дополнение к формату образца, количеству каналов и частоте дискретизации, `callback` данных и указатель пользовательских данных также устанавливаются через конфигурацию. Указатель пользовательских данных не передается в `callback` как параметр, а вместо этого устанавливается в член `pUserData` объекта `ma_device`, к которому вы можете обращаться напрямую, поскольку все структуры `miniaudio` прозрачны.

Инициализация устройства выполняется с помощью `ma_device_init()`. Это вернет код результата, сообщаящий вам, что пошло не так, если что-то. При успехе вернет `MA_SUCCESS`. После завершения инициализации устройство будет в остановленном состоянии. Чтобы запустить его, используйте `ma_device_start()`. Деинициализация устройства остановит его, что и делает приведенный выше пример, но вы также можете остановить устройство с помощью `ma_device_stop()`. Чтобы возобновить устройство, просто снова вызовите `ma_device_start()`. Обратите внимание, что важно никогда не останавливать или не запускать устройство изнутри `callback`. Это приведет к взаимной блокировке. Вместо этого вы устанавливаете переменную или сигнализируете событие, указывающее, что устройство нужно остановить, и обрабатываете это в другом потоке.

Следующие API никогда не должны вызываться внутри `callback`:

- `ma_device_init()`
- `ma_device_init_ex()`
- `ma_device_uninit()`
- `ma_device_start()`
- `ma_device_stop()`

Вы никогда не должны пытаться деинициализировать и переинициализировать устройство внутри `callback`. Вы также никогда не должны пытаться остановить и запустить его изнутри `callback`.

Приведенный выше пример демонстрирует инициализацию устройства воспроизведения, но это работает точно так же для захвата. Все, что вам нужно сделать, это изменить тип устройства с `ma_device_type_playback` на `ma_device_type_capture` при настройке конфигурации.

```
ma_device_config config = ma_device_config_init(ma_device_type_capture);
config.capture.format    = MY_FORMAT;
config.capture.channels  = MY_CHANNEL_COUNT;
```

В callback данных вы просто читаете из входного буфера (pInput в приведенном выше примере) и оставляете выходной буфер в покое (он будет установлен в NULL, когда тип устройства установлен в ma_device_type_capture).

Это доступные типы устройств и как вы должны обращаться с буферами в callback:

Тип устройства	Поведение Callback
ma_device_type_playback	Записывайте в выходной буфер, оставьте входной буфер нетронутым
ma_device_type_capture	Читайте из входного буфера, оставьте выходной буфер нетронутым
ma_device_type_duplex	Читайте из входного буфера, записывайте в выходной буфер
ma_device_type_loopback	Читайте из входного буфера, оставьте выходной буфер нетронутым

Вы заметите в приведенном выше примере, что формат образца и количество каналов указаны отдельно для воспроизведения и захвата. Это для поддержки различных форматов данных между устройствами воспроизведения и захвата в полнодуплексной системе. Примером может быть то, что вы хотите захватывать аудиоданные как монофонический поток (один канал), но выводить звук в стереосистему динамиков. Обратите внимание, что, если вы используете разные форматы между воспроизведением и захватом в полнодуплексной конфигурации, вам нужно будет конвертировать данные самостоятельно.

Приведенный выше пример не указывал физическое устройство для подключения, что означает, что он будет использовать устройство по умолчанию операционной системы. Если у вас подключено несколько физических устройств и вы хотите использовать конкретное, вам нужно будет указать ID устройства в конфигурации.

```
config.playback.pDeviceID = pMyPlaybackDeviceID;    // Only if requesting a
config.capture.pDeviceID = pMyCaptureDeviceID;      // Only if requesting a
config.capture, duplex or loopback device.
```

Чтобы получить ID устройства, вам нужно будет выполнить перечисление устройств, однако это требует использования новой концепции, называемой "контекстом". Концептуально контекст находится над устройством. Есть один контекст для многих устройств. Цель контекста - представлять бэкенд на более глобальном уровне и выполнять операции вне области отдельного устройства. В основном он используется для выполнения связывания во время выполнения с библиотеками бэкендов, инициализации бэкендов и перечисления устройств.

```
ma_context context;
if (ma_context_init(NULL, 0, NULL, &context) != MA_SUCCESS) {
    // Error.
```

```

}

ma_device_info* pPlaybackInfos;
ma_uint32 playbackCount;
ma_device_info* pCaptureInfos;
ma_uint32 captureCount;
if (ma_context_get_devices(&context, &pPlaybackInfos, &playbackCount,
&pCaptureInfos, &captureCount) != MA_SUCCESS) {
    // Error.
}

// Loop over each device info and do something with it. Here we just print the
// name with their index. You may want
// to give the user the opportunity to choose which device they'd prefer.
for (ma_uint32 iDevice = 0; iDevice < playbackCount; iDevice += 1) {
    printf("%d - %s\n", iDevice, pPlaybackInfos[iDevice].name);
}

ma_device_config config = ma_device_config_init(ma_device_type_playback);
config.playback.pDeviceID = &pPlaybackInfos[chosenPlaybackDeviceIndex].id;
config.playback.format = MY_FORMAT;
config.playback.channels = MY_CHANNEL_COUNT;
config.sampleRate = MY_SAMPLE_RATE;
config.dataCallback = data_callback;
config.pUserData = pMyCustomData;

ma_device device;
if (ma_device_init(&context, &config, &device) != MA_SUCCESS) {
    // Error
}

...

ma_device_uninit(&device);
ma_context_uninit(&context);

```

Первое, что мы делаем в этом примере, - это инициализируем объект `ma_context` с помощью `ma_context_init()`. Первый параметр - это указатель на список значений `ma_backend`, которые используются для переопределения приоритетов бэкендов по умолчанию. Когда это `NULL`, как в этом примере, используются приоритеты `miniaudio` по умолчанию. Второй параметр - это количество бэкендов, перечисленных в массиве, на который указывает первый параметр. Третий параметр - это указатель на объект `ma_context_config`, который может быть `NULL`, в этом случае используются значения по умолчанию. Конфигурация контекста используется для установки `callback` логирования, пользовательских `callback` выделения памяти, пользовательских данных и некоторых специфичных для бэкенда конфигураций.

После инициализации контекста вы можете перечислять устройства. В приведенном выше примере мы используем более простую функцию `ma_context_get_devices()`, однако вы также можете использовать `callback` для обработки устройств, используя `ma_context_enumerate_devices()`. При использовании `ma_context_get_devices()` вы предоставляете указатель на указатель, который при выводе будет установлен на указатель на буфер, содержащий список структур `ma_device_info`. Вы также предоставляете указатель

на беззнаковое целое число, которое получит количество элементов в возвращенном буфере. Не освобождайте возвращенные буферы, поскольку их память управляется внутренне `miniaudio`.

Структура `ma_device_info` содержит член `id`, который является ID, который вы передаете в конфигурацию устройства. Она также содержит имя устройства, которое полезно для представления списка устройств пользователю через UI.

При создании собственного контекста вы захотите передать его в `ma_device_init()` при инициализации устройства. Передача `NULL`, как мы делаем в первом примере, приведет к тому, что `miniaudio` создаст контекст для вас, чего вы не хотите делать, поскольку вы уже создали контекст. Обратите внимание, что внутри контекст отслеживается только по его указателю, что означает, что вы не должны изменять местоположение объекта `ma_context`. Если это проблема, рассмотрите использование `malloc()` для выделения памяти для контекста.

Кодирование

API `ma_encoding` используется для записи аудиофайлов. Единственным поддерживаемым выходным форматом является WAV. Это можно отключить, указав следующую опцию перед реализацией `miniaudio`:

```
#define MA_NO_WAV
```

Кодировщик может быть инициализирован для записи в файл с помощью `ma_encoder_init_file()` или из данных, доставляемых через `callback`, с помощью `ma_encoder_init()`.

```
FORMAT, CHANNELS, SAMPLE_RATE);
ma_encoder encoder;
ma_result result = ma_encoder_init_file("my_file.wav", &config, &encoder);
if (result != MA_SUCCESS) {
    // Error
}
```

...

```
ma_encoder_uninit(&encoder);
```

При инициализации кодировщика вы должны указать конфигурацию, которая инициализируется с помощью `ma_encoder_config_init()`. Здесь вы должны указать тип файла, выходной формат образца, количество выходных каналов и выходную частоту дискретизации. Поддерживаются следующие типы файлов:

Перечисление	Описание
<code>ma_encoding_format_wav</code>	WAV

Если формат, количество каналов или частота дискретизации не поддерживается выходным типом файла, будет возвращена ошибка. Кодировщик не будет выполнять

преобразование данных, поэтому вам нужно будет преобразовать их перед выводом любых аудиоданных. Для вывода аудиоданных используйте `ma_encoder_write_pcm_frames()`.

```
ma_uint64 framesWritten;
result    = ma_encoder_write_pcm_frames(&encoder,      pPCMFramesToWrite,
framesToWrite, &framesWritten);
if (result != MA_SUCCESS) {
    ... handle error ...
}
```

Переменная `framesWritten` будет содержать количество кадров PCM, которые были фактически записаны. Это необязательно, и вы можете передать `NULL`, если вам это не нужно.

Кодировщик должен быть деинициализирован с помощью `ma_encoder_uninit()`.

Задание на работу

1. Ознакомьтесь с документацией по библиотеке **miniaudio** на официальном сайте <http://miniaud.io/>
2. Скачайте библиотеку и создайте консольное приложение для захвата и воспроизведения звука со звукового устройства по умолчанию вашего компьютера с использованием низкоуровневого API на основе примера из методических указаний.
3. На основе примера и документации напишите программу для захвата и сохранения звука на компьютере в виде файла формата WAV.

Программа захвата звука

В программе демонстрируется захват звука микрофона и воспроизведение его через звуковое устройство.

В примере звук захватывается с микрофона, выбранного в системе по умолчанию, и воспроизводится на устройстве по умолчанию.

```
#include "miniaudio.c"
#include <stdio.h>
#include <string.h>
#ifdef __EMSCRIPTEN__
void main_loop__em()
{
}
#endif

void data_callback(ma_device* pDevice, void* pOutput, const void* pInput, ma_uint32
frameCount)
{
    /* This example assumes the playback and capture sides use the same format and
channel count. */
    if (pDevice->capture.format != pDevice->playback.format || pDevice-
>capture.channels != pDevice->playback.channels) {
        return;
    }
}
```



```

    /* In this example the format and channel count are the same for both input and
    output which means we can just memcpy(). */
    memcpy(pOutput, pInput, frameCount * ma_get_bytes_per_frame(pDevice-
>capture.format, pDevice->capture.channels));
}

int main(int argc, char** argv)
{
    ma_result result;
    ma_device_config deviceConfig;
    ma_device device;

    deviceConfig = ma_device_config_init(ma_device_type_duplex);
    deviceConfig.capture.pDeviceID = NULL;
    deviceConfig.capture.format = ma_format_s16;
    deviceConfig.capture.channels = 2;
    deviceConfig.capture.shareMode = ma_share_mode_shared;
    deviceConfig.playback.pDeviceID = NULL;
    deviceConfig.playback.format = ma_format_s16;
    deviceConfig.playback.channels = 2;
    deviceConfig.dataCallback = data_callback;
    result = ma_device_init(NULL, &deviceConfig, &device);
    if (result != MA_SUCCESS) {
        return result;
    }

#ifdef __EMSCRIPTEN__
    getchar();
#endif

    ma_device_start(&device);

#ifdef __EMSCRIPTEN__
    emscripten_set_main_loop(main_loop__em, 0, 1);
#else
    printf("Press Enter to quit...\n");
    getchar();
#endif

    ma_device_uninit(&device);

    (void)argc;
    (void)argv;
    return 0;
}

```

Содержание отчета

1. Цель работы.
2. Задание на работу.
3. Исходный код и скриншот консоли с информацией о созданном файле.
4. Выводы по работе.